

Luna Brown

Professor Stephen France

ECE 49595 SD1

26 Mar 2026

Individual Exercise: Design Alternatives

Generative AI was not used to author any portions of this document.

1. Select and Define your System Element

I will be performing my design trades study on the skill detection subsystem of our system. The role of this element is to intake resumes from user input and job listings from the web scraper, identify skills the applicant holds or the job listing requires, determine how proficient an applicant is in that skill or how important a certain skill is for a listing, and store that information in a database for use by the job recommendation system. This subsystem is vital for the accuracy of job recommendations and is the main way that applicants will be able to find relevant jobs. While applicants will be able to edit their skills and proficiency levels if this system performs its task erroneously, most job listings are expected to be automatically added from the webscraper, so the importance of correctly extracting job requirements in a searchable way is high.

For these reasons, accuracy and level of detail are the most critical design drivers for this subsystem. Accuracy of extracted skills is vital for reducing the probability that an applicant is recommended jobs they are wholly unqualified for or which do not match their field of expertise, and level of detail (as in, detail beyond just the skill names that appear in the text) is important for ordering jobs in similar fields by relevance to the user, determining the percentile ranking of a candidate's qualifications for a job versus other applicants, and ensuring a user meets the appropriate level of qualification required of a job before they are recommended it by the system. Maintainability is another critical design driver: the terms used for skills may evolve over time, and new skills will be created entirely, and the system should be resilient to that without the need for human interference. Cost and performance are lesser concerns, but they are also drivers of this design: while the extraction system will run in the background and not impair performance of regular user activities, we are still expecting to handle a large number of job listings. As such,

the time and monetary cost of extracting skills from individual listings should be kept to a minimum so that the service is available in a timely manner and is not prohibitively expensive.

The following functional requirements are impacted by this subsystem:

- **FR-001 Accurate Job Recommendations:** The accuracy of job recommendations is dependent on the accuracy of searchable/sortable information extracted from job listings and resumes. As such, this functional requirement places an accuracy requirement on the skill detection subsystem.
- **FR-004 ChatGPT Hallucination Minimization:** This requirement states that skill matches should be verifiably accurate to the description, as measured by sending a sample of listings and resumes through the system and verifying the end results via human observation. As such, the impact of false positives and negatives in skill detection must be minimized
- **FR-006 Skill Gap Detection:** This requirement states that the system must detect when an applicant is missing proficiency in one or more skills that are required for a listing, so this subsystem must accurately determine skills required for a listing and skills held by a candidate.
- **NFR-001 Fast Webpage Performance:** While the performance of this subsystem does not directly impact page load times, the tagging of listings and applicants with skills is vital to recommendation requests being performant. Essentially, this just means that this subsystem must exist and cannot be replaced with, say, an LLM directly matching compatibility between users and jobs.
- **NFR-004 - Data Storage:** This NFR requires job recommendations to be performant even under a high number of stored listings and skills. As such, our subsystem must store data in a way that scales well and allows for performant searching of the job listing database.

2. Establish Evaluation Criteria

Criterion	Description	Weight	Rationale
C-001 False Positive Resilience	The ability of the system to avoid identifying skills in a resume or job listing that are not present	25%	This is informed by FR-004 and the general need for skill accuracy. Weighted equally to false negatives since they have a similar impact on accuracy. Since accuracy is the most important design feature, they should both add to 50%

C-002 False Negative Resilience	The ability of the system to identify all skills present in a resume or job listing.	25%	This is informed by FR-006 and the general need for skill accuracy. Weighted equally to false positives since they have a similar impact on accuracy. Since accuracy is the most important design feature, they should both add to 50%
C-003 Maintainability	The ability of the system to remain accurate and detailed without human interference as the job market evolves and new skills are created	15%	This is required for the accuracy of our system in the long term. Level of detail is slightly more important due to it being a requirement for accurate recommendations regardless of time period, however.
C-004 Level of Detail	The ability of the system to provide information beyond what skills are present in the listing or resume, such as proficiency level or certification requirements	20%	This is informed by FR-001 and level of detail being a design driver. Because this heavily impacts the accuracy of recommendations, but to a lesser degree than skill detection accuracy, this should be weighted to be around 40% as important as accuracy.
C-005 Cost Efficiency	The ability of the system to identify skill information in a cost-efficient manner	8%	This is informed by the large amount of data we are processing and the need to fit within the project's budget. Slightly more important than performance efficiency since they will tend to be proportional to each other.
C-006 Performance	The ability of the system to identify skill information in a timely manner	7%	This is informed by the large amount of data we are processing and the need for it to be processed before the deadline. Slightly less important than cost efficiency as stated above.

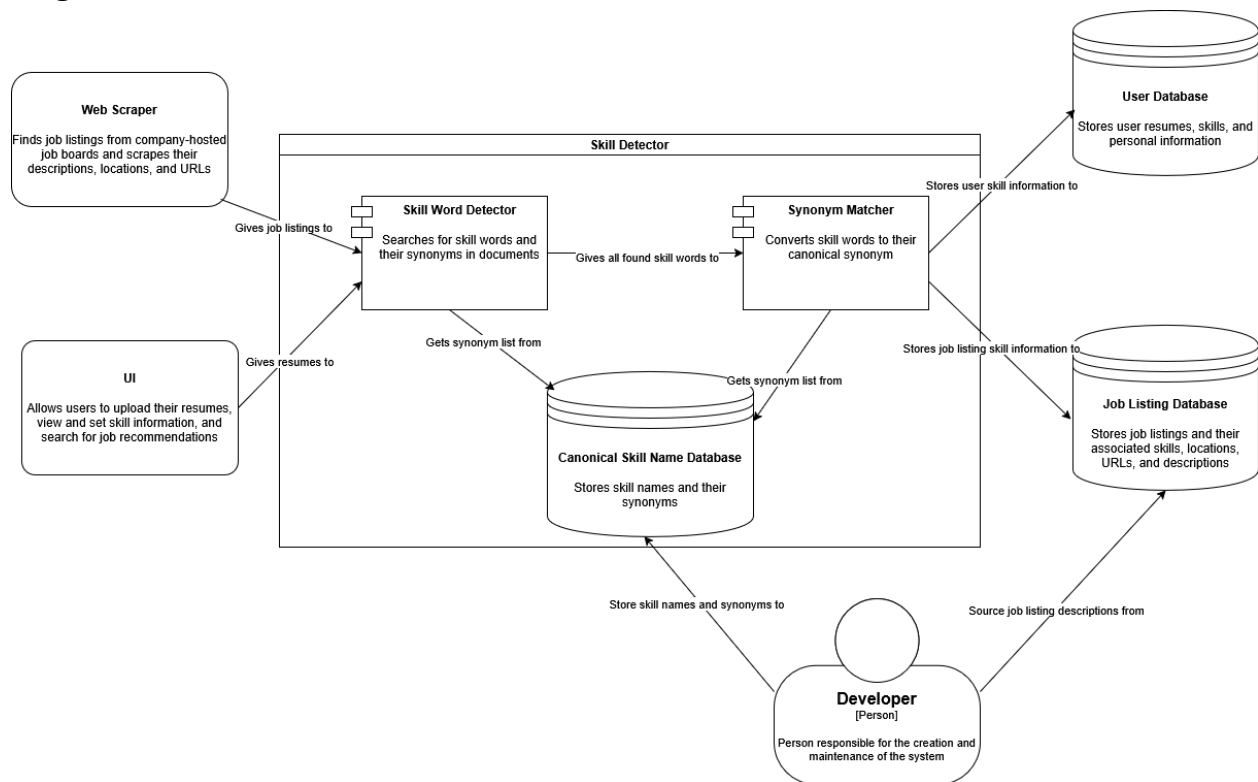
3. Brainstorm Designs

D-001 Static Skill Repository with Word Matcher

Description: A list of skills is generated by taking a representative sample of job listings as they appear today, then using humans and/or LLMs to parse out skill names from them. These skills are then grouped into synonym groups and assigned a canonical, representative member for the sake of storage. The synonym group is stored for reference. Then, when the system is given a resume or job listing to pull skill information from, it searches the document for any words contained in the canonical skill list or their synonyms and assigns the canonical names for these words as skills.

This design is considered for its simplicity and low overhead. While generating the initial skill repository is an up-front time sink, searching through this skill repository and matching resumes and listings against it is fast and requires no complex processing. Its deterministic nature also eliminates false positives and reduces the chances for false negatives.

Diagram:



Pros:

- False positives can only occur for homonyms (i.e. “programming” in the sentence “programming a robot” versus “creating TV programming”), which can largely be solved

by smartly choosing canonical skill names and synonym lists to be as specific as possible. In essence, this makes false positives very rare.

- Skill detection is fast and cost-efficient due to the low computational intensity of word matching.
- Degrees and certificates can be matched for via including them in the skill database.

Cons:

- Additional info like skill proficiency level is impossible to determine using this system
- Niche certifications are not likely to be present in the skill database, so they will not be included as requirements for jobs that need them.
- This system is not resilient to changes in terminology or new skills being created over time. Developers will have to redo the process of creating the skill database on a yearly basis to ensure evolutions in technology and society are accounted for.
 - While LLMs could theoretically be used to automate this process, if there is no human verification step involved, we will lose the benefits of choosing smart canonical skills names/synonym lists and the deterministic nature of word matching, which is a net negative.
- Strict word matching will lead to false negatives when uncommon synonyms or phrasings are used for certain skills

D-002 Snapshotted Skill Repository with LLM Matcher

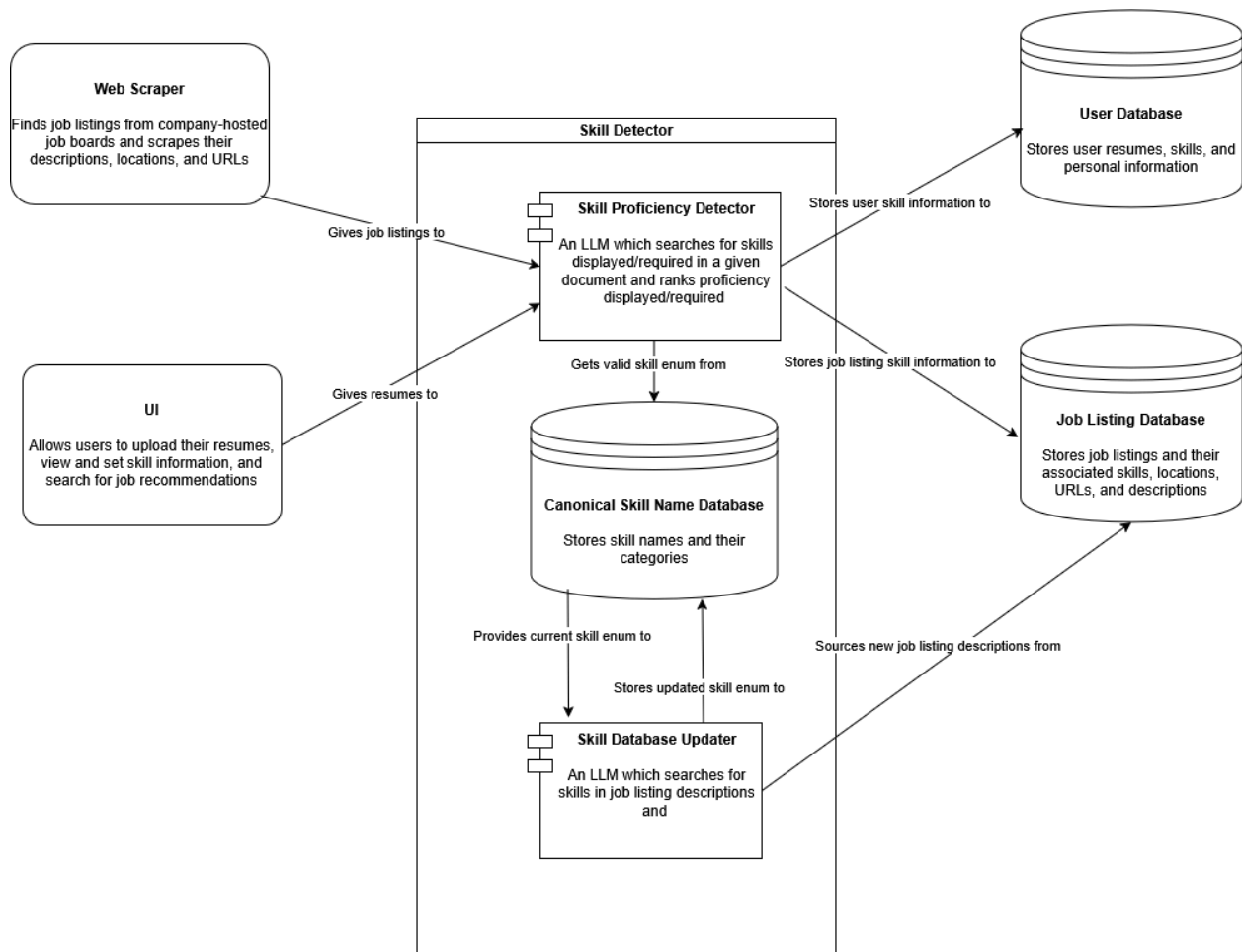
Description: A list of skills is generated by taking a representative sample of job listings as they appear today, then using LLMs to parse out skill names from them. These skills are then grouped into synonym groups and assigned a canonical, representative member for the sake of storage. Only the canonical skill name is stored, and each skill is given a broad category for searching purposes. On a monthly basis, this process is repeated using job listings stored in the database. Job listings used for this purpose are then marked so that they are not included in this process again.

Then, when the system is given a resume or job listing to pull skill information from, an LLM is provided the document and a list of skill names, then asked to mark any skills the document mentions and rate the proficiency the applicant has (or listing requires) for each skill.

This design is considered due to its ability to dynamically update the skill list as the job market changes and new skills are created. As opposed to the static repository of D-001, it should be more maintainable and require less human intervention. While including LLMs in the process of skill repository generation may result in hallucinations, the effects of hallucinated skills in the database is minimal, so this is not a major concern. However, as a result of the change to a snapshotted skill repository, a simple word matcher becomes non-functional. The skill names generated by the LLM creating the skill database are not guaranteed to be smartly

chosen to avoid conflicts with synonyms (and may be hallucinated entirely), so a word matcher is likely to have a greater rate of false positives and negatives in this scenario. As such, an LLM must be used to parse which skills make sense to tag given the context of the document.

Diagram:



Pros:

- Snapshotted skill database retains accuracy in response to change as opposed to a static one
- Automated skill database updates reduces the need for human involvement in the process
- Using an LLM for skill detection from documents allows context to be included in the decision process, which will reduce false negatives and certain classes of false positives compared to a word matcher.
- Niche degrees, certifications, and skills are more likely to appear in the skill database due to iterating over all scraped job listings.
- Skill proficiency level and requirements are able to be determined.
- A static list of job skills minimizes the chances of LLMs hallucinating skills that don't exist and assigning them to documents.

Cons:

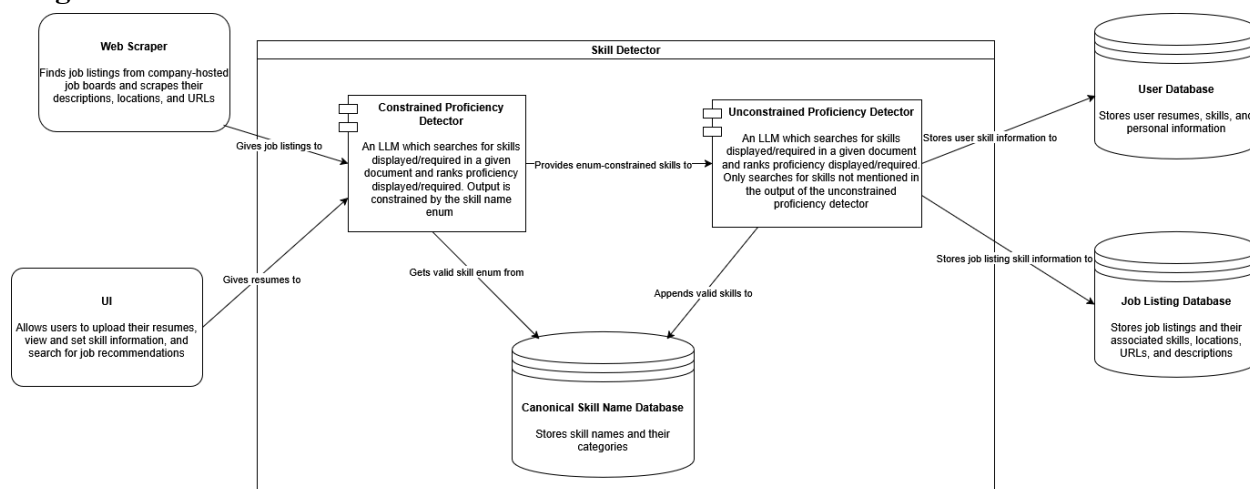
- LLMs are more likely to hallucinate skills from job listings and resumes, which will lead to false positives.
- Skill detection is slow and costly per document due to the use of an LLM. This is less of an issue for job listings since they can be batched effectively, but it is still worse than word matching
- Generating the skill database is costly due to the use of LLMs. This can be mitigated by only analyzing new jobs in the database, but it is still an issue considering the frequency at which it must happen to maintain relevancy.
- Constraining the skills assigned to jobs and resumes to a periodically-updated list means that listings and resumes with new skills will not be properly assigned those skills until the update happens, which could take a very long time based on the chosen update period.

D-003 Dynamic Skill Repository with LLM Matcher

Description: An initial list of skills and corresponding categories is created from a sample of job listings through human-guided LLMs. Then, when a document is sent to the subsystem for skill detection, an LLM is given the existing list of skills and the document to extract skills from. It is then asked to list which skills and certificates from the existing list the document exhibits/requires, as well as their level of proficiency. That sublist is then given to a separate LLM alongside the original document. It is then asked what skills and certificates are exhibited/required by the document that are not included in that sublist, and the skill repository is then updated with the skills identified by this second LLM. Finally, the combined skill list is stored alongside the resume/listing.

This design is considered because it eliminates the long turnaround times that a snapshotted list suffers from. When a new skill is invented, it is immediately added to the dataset, there is no downtime period. In rapidly-changing fields like software engineering, this is highly beneficial to skill detection accuracy. It also avoids the long and costly process of regenerating the skill database on a periodic basis.

Diagram



Pros:

- A dynamic skill database retains accuracy in response to change as opposed to a static one.
- A dynamic skill database reduces the need for human involvement in the process
- Compared to a snapshotted database, a dynamic skill database can include newly-invented or niche skills faster.
- Using an LLM for skill detection from documents allows context to be included in the decision process, which will reduce false negatives and certain classes of false positives compared to a word matcher.
- Skill proficiency level and requirements are able to be determined.

Cons:

- LLMs are more likely to hallucinate skills from job listings and resumes, which will lead to false positives.
- Skill detection is slow and costly per document due to the use of an LLM. This is less of an issue for job listings since they can be batched effectively, but it is still worse than word matching. This cost is also greater than a snapshotted database due to the usage of multiple LLM calls per document.
- Any false negatives in the database-constrained LLM call may result in the database-updating LLM call adding a synonym to the database. While this failure mode should be rare, it can still lead to a snowball effect of more false negatives in the future. This could be mitigated through having a separate LLM periodically cull the still database of synonyms, but it is still an issue.

4. Complete the Trade Study

Evaluation Rankings (out of 10)

Design	False Positive Resilience	False Negative Resilience	Maintainability	Level of Detail	Cost Efficiency	Performance
D-001	10	3	1	4	10	10
D-002	5	8	10	10	3	3
D-003	5	10	10	10	1	1

Final Scores:

Design	Total (out of 10)
D-001	5.7
D-002	6.7
D-003	7.4

In the end, I have decided to use the dynamic skill repository with LLM matcher. While it is the least cost-performant and most time-consuming option available, it also provides the best accuracy while providing the greatest level of detail and requiring the least human involvement during maintenance. While some aspects of its design could lead to false negatives, I believe that it is an overall net gain compared to D-002 and can be mostly remedied through automated, periodic processes. While the snapshot skill repository is still preferable to a static one for maintainability reasons, it does not substantially improve on accuracy, which is why D-003 is more preferable.